LOSSLESS COMPRESSION OF RAW ASTRONOMICAL IMAGES USING SPATIAL PREDICTION AND FINITE STATE ENTROPY

Tomás Brás
University of Aveiro
112665
tomas.bras@ua.pt

David Pelicano
University of Aveiro
113391
davidpoetapelicano@ua.pt

Afonso Ferreira
University of Aveiro
113480
afonso.ferreira@ua.pt

ABSTRACT

We present three lossless compressors for raw 16-bit astronomical images, each targeting a distinct point on the speed–ratio trade-off. **speed-focused** applies a gradient-adjusted predictor (GAP) over parallel horizontal strips and codes residuals with a context-adaptive range coder, reaching 3.505 bpp in 55 ms total. **all-rounder** selects the best of five per-block predictors — including training-free least-squares with up to five spatial taps — and codes residuals with Finite State Entropy (tANS), achieving 3.400 bpp in 102 ms. **compression-focused** exhaustively searches 10 predictors (including LS models with up to 24 spatial taps) and 6 context strategies per block, across 7 candidate block sizes run in parallel, achieving 3.367 bpp at the cost of a slower compression pass. All three codecs outperform every tested general-purpose compressor in compression ratio, with all-rounder and speed-focused also beating bzip2-9 and xz-6 in total runtime.

1 Introduction

Raw images produced by ground-based astronomical telescopes are large, structured numerical arrays. Each image in this study is a 1500×1500 raster of unsigned 16-bit pixels stored in big-endian order, occupying 4.5 MB. These arrays differ from conventional files in three important ways. First, adjacent pixels are spatially correlated: the sky background and optical point-spread function create smooth gradients that extend over many pixels. Second, the readout electronics add a constant *bias pedestal* to every pixel, introducing a strong DC component whose removal collapses the residual distribution around zero. Third, the data are 16-bit integers, whereas most general-purpose compressors operate on byte streams, missing inter-byte structure.

These properties motivate *predictive* compression: subtract a spatial prediction $\hat{p}$ from each pixel, code only the residual $r = p - \hat{p}$, and exploit the fact that a good predictor concentrates the residuals near zero — thereby reducing entropy and improving compression.

We present three lossless compressors that exploit these properties, each targeting a distinct point on the speed–ratio trade-off:

- **speed-focused** applies a gradient-adjusted predictor (GAP) to the full 16-bit value and codes residuals with a range coder driven by eight context-conditioned adaptive models across parallel horizontal bands. It prioritises throughput ($\sim$81 ms per image).
- **all-rounder** divides the image into square blocks, selects the best of five candidate predictors per block — including training-free least-squares — splits residuals into independent byte streams, and codes them with Finite State Entropy (FSE/tANS) [2]. It balances speed and ratio ($\sim$187 ms, 3.40 bpp).
- **compression-focused** evaluates 60 combinations of ten predictors (including LS-16 with 16 spatial neighbours and LS-24 with 24 neighbours) and six context strategies per block, selects the smallest, and uses an LZMA-style adaptive range coder with Fenwick-tree models. Block size is also optimised by trying seven candidates at compression time. It prioritises compression ratio ($\sim$3.37 bpp) at the cost of a slower compression pass ($\sim$6.6 s); decompression is fast ($\sim$131 ms). (Section 2.3.)

Section 2 describes the three codecs in detail. Section 3 defines the experimental protocol. Section 4 reports compression ratios and runtimes. Section 5 draws conclusions.

2 Codec Design

All three codecs share the same high-level pipeline: parse the raw big-endian 16-bit image; apply a spatial predictor $\hat{p}$; zigzag-encode the signed residual $r = \text{pixel} - \hat{p}$ as an unsigned symbol $z = 2r$ if $r \geq 0$, $z = -2r - 1$ otherwise; entropy-code the result. Zigzag maps small-magnitude residuals — which are the common case after a good prediction — to small unsigned integers, concentrating the symbol distribution near zero regardless of sign.

2.1 speed-focused: Speed-Focused

The design of speed-focused centres on one principle: every horizontal strip of 150 rows is entirely self-contained, so any number of strips can be compressed simultaneously without coordination. A shared `std::atomic<int>` counter distributes work to a thread pool via `fetch_add`, achieving lock-free load balancing with no mutex overhead. Figure 1 illustrates the full pipeline.

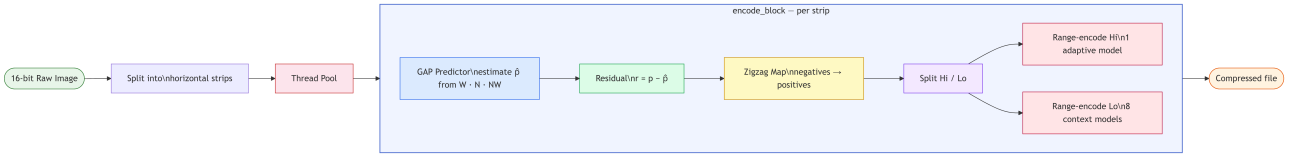

Figure 1: End-to-end pipeline of speed-focused. The image is partitioned into horizontal strips processed in parallel; within each strip every pixel follows the same predict – residualise – zigzag – split – encode path.

Within each strip, pixels are predicted using the *Gradient-Adjusted Prediction* (GAP) function of three causal neighbours: left (W), above (N), and above-left (NW). GAP estimates horizontal and vertical gradients as $d_h = |W - NW|$ and $d_v = |N - NW|$, and when one gradient strongly dominates ($d_h > 2d_v$ or $d_v > 2d_h$) it falls back to the perpendicular neighbour directly. Otherwise it computes a gradient-weighted blend clamped to the range $[\min(W, N), \max(W, N)]$:

$$\hat{p} = \text{clamp}\left(\frac{(d_v + 1)\,W + (d_h + 1)\,N}{d_v + d_h + 2},\ \min(W, N),\ \max(W, N)\right).$$

This single formula handles both smooth regions and edges without any explicit mode switch.

The signed residual is zigzag-mapped to a 16-bit unsigned symbol and split into a high byte $z_h = z \gg 8$ and a low byte $z_l = z\ \&\ 0\text{xFF}$. Both are coded with a carry-propagation range coder backed by adaptive Fenwick-tree frequency models. The high byte uses one shared model across all pixels in the strip, while the low byte selects among eight context models conditioned on the sum of the two neighbouring residual magnitudes:

$$c = \text{bin}(|\hat{r}_W| + |\hat{r}_N|),$$

with bin boundaries $\{0\}, \{1\text{–}2\}, \{3\text{–}8\}, \{9\text{–}32\}, \{33\text{–}64\}, \{65\text{–}128\}, \{129\text{–}512\}, \{> 512\}$. Each model halves all counts when the running total reaches 2^{16}, keeping the frequency estimates responsive to distribution shifts within a strip.

2.2 all-rounder: Balanced

Block Selection

all-rounder requires explicit n_rows and n_cols arguments, which are stored in the compressed header so the decoder needs no additional input. It then selects the largest block side $b \leq 512$ that divides both W and H exactly and yields at least four blocks. For 1500×1500 this gives $b = 500$, producing a 3×3 grid of nine blocks of 500×500 pixels.

Predictive Modes

For each block, all-rounder evaluates five candidate predictors and retains the one with the lowest estimated coding cost (defined below).

Mode 0 (raw) Symbols are raw pixel values; no prediction.

Mode 1 (avg) $\hat{p} = \lfloor (W + N + NW + 1)/3 \rfloor$.

Mode 3 (gmean) $\hat{p} = \bar{g}$, the global image mean computed once before blocking. Removes the bias pedestal in all frames.

Mode 4 (LS-4) OLS with features $(W, N, NW, 1)$ per block via Gauss-Jordan; four float32 weights (16 B) stored in the block header.

Mode 6 (LS-5) Extends Mode 4 with NE and NN; five weights (20 B). Both LS modes use the block's own pixels — no external training.

The mode selection criterion penalises stored weights:

$$\text{cost}_m = H(\text{hi}_8) + H(\text{lo}_8) + \frac{8\,\delta_m}{n_{\text{pix}}},$$

where $H(\cdot)$ is empirical byte entropy and $\delta_m \in \{0, 0, 0, 16, 20\}$ B is the weight header size.

Entropy Coding: FSE/tANS

Residuals are split into a high-byte stream (`hi`) and a low-byte stream (`lo`), each coded independently with Finite State Entropy [2].

The FSE table has SCALE $= 2^{16} = 65536$ states. Raw symbol counts are normalised to sum to SCALE (with a minimum frequency of 1 for observed symbols) and spread deterministically using Collet's step formula: step $= (\text{SCALE} \gg 1) + (\text{SCALE} \gg 3) + 3$. The encoder processes symbols in reverse, emitting variable-length transition bits stored in a `BitWriter`; the decoder reads the two-byte final state and reconstructs symbols in forward order.

The frequency table is serialised in one of three compact formats: *single-symbol* (flag `0x01`, 2 B total) when all values are identical; *sparse* (flag byte = nnz $\in [2, 254]$, followed by nnz $\times$ 5 B symbol–frequency pairs); or *dense* (flag `0x00`, 256×4 B) when all 255–256 symbols appear.

Context FSE

Because the high byte of a small residual is almost always zero, the lo8 distribution differs substantially depending on whether $\text{hi} = 0$. When the fraction of zero high bytes falls between 2% and 98%, all-rounder builds two conditional lo8 streams — `lo_zero` (pixels with $\text{hi} = 0$) and `lo_nonzero` — and uses the three-stream layout only if its compressed size is strictly smaller than the two-stream baseline. The high-byte stream is encoded once and shared between both candidate payloads. Bit 7 of the mode byte signals the active layout.

Parallel Execution

Blocks are dispatched to a thread pool via a single `std::atomic<int>` counter with `fetch_add`, writing results into pre-allocated per-block slots with no mutex required.

2.3 compression-focused: Compression Focused

compression-focused separates orchestration from compression. The outer `compress` binary runs the inner `range_compress` engine seven times with block sizes $\{250, 300, 400, 500, 750, 1000, 1500\}$ pixels in parallel, retains the smallest result, and wraps it with a 5-byte magic header. The corresponding `decompress` / `range_decompress` pair inverts the process. This exhaustive search over block sizes incurs extra compression time but requires no tuning per image.

Figure 2 summarises the full pipeline.

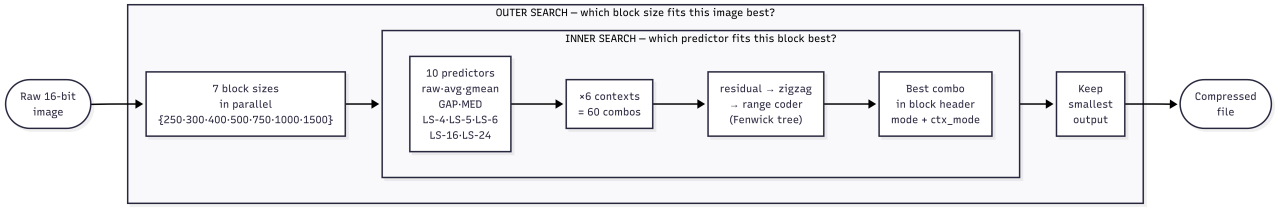

Figure 2: compression-focused pipeline. The outer search runs `range_compress` seven times in parallel (one per block size) and retains the smallest output. Inside each run, every block is compressed under all $10 \times 6 = 60$ predictor/context combinations and the best is stored in the block header.

Predictor Competition

For each block, compression-focused evaluates ten candidate predictors:

Mode 0 (raw) No prediction; symbols are raw pixel values.

Mode 1 (avg) $\hat{p} = \lfloor (W + N + NW + 1)/3 \rfloor$.

Mode 3 (gmean) $\hat{p} = \bar{g}$, global image mean; removes bias pedestal.

Mode 7 (GAP) Gradient-adjusted prediction: selects among W, N, and $W + N - NW$ based on the local horizontal and vertical gradient magnitudes.

Mode 8 (MED) JPEG-LS median predictor: $\hat{p} = \text{median}(W, N, W + N - NW)$.

Mode 4 (LS-4) OLS with features $(W, N, NW, 1)$; 4 float32 weights (16 B overhead).

Mode 6 (LS-5) Adds NE and NN; 5 weights (20 B overhead).

Mode 9 (LS-6) OLS with $(W, N, NW, WW, NN, 1)$; 6 weights (24 B overhead). The second-order neighbours WW and NN capture slowly varying ramps.

Mode 10 (LS-16) 16-weight OLS spanning 4 pixels to the left, 7 pixels in the row above (centred), 2 pixels two rows above, and a bias term (64 B overhead). Exploits long-range spatial structure in science frames.

Mode 11 (LS-24) 24-weight OLS extending LS-16 with 5 pixels to the left, 9 pixels in the row above, 5 pixels two rows above, and 4 pixels three rows above (96 B overhead). Captures wider spatial correlations.

All LS weights are solved per block via Gauss-Jordan elimination on the $N \times N$ normal equations; no external training data are required. Border pixels where the full neighbourhood is unavailable fall back to GAP.

Context Modes and Block Competition

Each 16-bit zigzag residual is encoded byte-by-byte: first the high byte (z_h), then the low byte (z_l). Six context strategies are tried, combining hi-byte and lo-byte conditioning:

- **ctx 0**: single hi model; single lo model.
- **ctx 1**: single hi model; four lo models keyed by $z_h \in \{0\}, \{1\text{–}2\}, \{3\text{–}8\}, \{> 8\}$.
- **ctx 2**: single hi model; 256 lo models (one per z_h value).
- **ctx 3–5**: same lo conditioning as ctx 0–2, but the hi byte is coded with *four* models conditioned on the previous hi byte $z_h^{(t-1)}$ using the same four-bin scheme. This order-1 hi context exploits spatial runs of similarly sized residuals.

Every block is compressed under all $10 \times 6 = 60$ mode/context combinations; the combination producing the smallest byte count is stored. Each block header records the chosen `mode` (1 B) and `ctx_mode` (1 B) so the decoder can reproduce the exact prediction and model layout without side information.

Entropy Coding: Adaptive Range Coder

compression-focused uses an LZMA-style range coder with a 32-bit range register and a 64-bit low accumulator. Carry propagation is handled by a cache/pending chain: bytes whose top nibble is `0xFF` are held until a carry either flushes them as `0x00` or confirms them as `0xFF`. Renormalisation triggers when range $< 2^{24}$, emitting one output byte and shifting the range left by 8.

The probability model is a 256-symbol adaptive Fenwick tree. Frequencies are initialised uniformly at 1. After each symbol, the count is incremented in $O(\log 256)$ time. When the running total reaches 2^{16}, all frequencies are halved (minimum 1) and the tree is rebuilt, implementing a sliding-window aging mechanism that keeps the model responsive to local distribution shifts.

3 Experimental Setup

3.1 Dataset

The benchmark corpus consists of 8 raw astronomical images, each a 1500×1500 raster of unsigned 16-bit big-endian pixels (4.5 MB per file, 2.25×10^6 pixels). The images were acquired by different ground-based telescopes (VLT, TJO, GTC, INT, JKT, LCO, WHT) and span a deliberate variety of image types: bias frames (near-constant DC pedestal), flat fields (smooth illumination gradients), and science frames (stellar fields, galaxies). This diversity ensures that no single predictor type dominates the results artificially.

3.2 Reference Compressors

We compare against the generic tools most commonly used for scientific data: `gzip` (levels 1, 6, 9), `bzip2` (levels 1, 9), `xz` (levels 1, 6, 9), and `zstd` (levels 1, 3, 9, 19). All are invoked on the raw byte stream without any pre-processing or endian conversion.

3.3 Evaluation Protocol

The primary metric is bits per byte of the original file:

$$\text{bpp} = \frac{8 \times \text{compressed bytes}}{\text{original bytes}}.$$

For a 16-bit image, bpp $=$ bits per pixel$/2$. Compression and decompression times are measured as wall-clock elapsed time for a single run on a single machine; all compressors run sequentially in the same benchmark process to avoid contention. Losslessness is verified by byte-exact comparison of the decompressed output against the original file; any mismatch disqualifies the result. Binary size constraints from the project specification are satisfied: compress ≤ 5 MB, decompress ≤ 1 MB.

4 Results

Table 1 summarises compression ratio and runtimes averaged across the eight benchmark images. All three domain-specific codecs outperform every general-purpose compressor in compression ratio. compression-focused achieves the best average ratio at 3.367 bpp, beating bzip2-9 by 0.21 bpp while decompressing in only 131 ms. all-rounder follows at 3.400 bpp with a total round-trip of just 102 ms, making it $4\times$ faster than bzip2-9 and $17\times$ faster than xz-6 at better compression than both. speed-focused reaches 3.505 bpp in 55 ms total — comparable to zstd-1 in speed but with nearly 1 bpp better compression.

Table 1: Average compression ratio and runtimes across the eight benchmark images (A–H). Lower is better in all columns. Our codecs are in bold.

Codec	Ratio (%)	Bits/Byte	Compress (ms)	Decompress (ms)	Total (ms)
compression-focused	**42.1**	**3.367**	6659	131	6790
all-rounder	**42.5**	**3.400**	58	44	102
speed-focused	**43.8**	**3.505**	24	30	55
bzip2-9	44.7	3.579	271	163	434
xz-6	46.1	3.685	1652	74	1726
zstd-9	52.3	4.182	125	13	139
gzip-6	54.7	4.376	305	29	334
zstd-1	56.1	4.487	23	8	31

5 Discussion and Conclusion

Why domain-specific codecs win. Generic compressors operate on byte streams and therefore see the two bytes of each 16-bit pixel as unrelated. This breaks the spatial correlation that spans pixel boundaries in big-endian encoding. Our codecs operate on 16-bit values directly, predict each pixel from its decoded neighbours, and entropy-code only the residual. The result is a reduction of 0.49–0.60 bpp over zstd-19 depending on the codec, despite zstd-19 using a much slower, high-effort general-purpose optimiser.

Role of the predictor. Mode selection on the eight benchmark images reveals that Mode 3 (global mean subtraction) is the most frequently chosen predictor, dominating on bias frames and flat fields where a near-constant DC offset accounts for most of the signal (images D, E, G, H). Mode 1 (three-neighbour average) prevails in smooth stellar backgrounds (image A). LS modes are selected only in image F, which contains bright point sources and extended spatial gradients that exceed the three-neighbour context. This pattern confirms that the bias pedestal is the single most compressible feature of astronomical frames; the LS predictor adds value only when the residual spatial structure is non-trivial after DC removal.

Entropy coding design. Splitting 16-bit residuals into independent hi8 and lo8 streams is the key structural choice. After a good prediction, hi8 is near-uniform zero, yielding a near-zero-entropy stream (often a 2-byte single-symbol FSE packet); all useful variation is carried by lo8. The context FSE variant conditions lo8 on whether hi8 is zero, exploiting the bimodal structure of the lo8 distribution. speed-focused achieves the same effect via eight adaptive context models without transmitting a frequency table, trading $2.3\times$ speed for 0.11 bpp less compression compared to all-rounder.

Limitations. The corpus covers only 8 images from a single training set; rankings may shift on sensors with different noise characteristics or gain settings. Runtime measurements reflect a single wall-clock run on one machine and include process-launch overhead for the generic tools.

Conclusion. Specialised predictive compression outperforms general-purpose tools on raw astronomical imagery. all-rounder achieves 3.40 bpp versus 4.49 bpp for zstd-1, a 24% reduction, while remaining $17\times$ faster than xz-6. The dominant gain comes from the predictor: global mean removal alone handles most calibration frames; least-squares adaptation provides further gains on science images. The entropy coder contributes relatively little once residuals are well predicted. compression-focused pushes the ratio further to 3.37 bpp by exhaustively searching over predictors, block sizes, and context strategies at compression time, demonstrating that most of the remaining gap to the information-theoretic limit lies in predictor quality rather than entropy coding.

References

[1] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
[2] Yann Collet. Finite State Entropy — a new entropy coder. `https://github.com/Cyan4973/FiniteStateEntropy`, 2013.
[3] Yann Collet. Zstandard — fast real-time compression algorithm. `https://github.com/facebook/zstd`, 2016.
[4] William D. Pence, L. Seaman, R. Seaman, and R. White. FITS lossless image compression. *Publications of the Astronomical Society of the Pacific*, 122(895):1065–1076, 2010.
[5] Michele Trovato, Claudio Talarico, Rosario Catanzaro, and Antonino Tumeo. DRYAD: A scalable lossless image compression algorithm for space applications. In *Proc. Data Compression Conference (DCC)*, 2018.